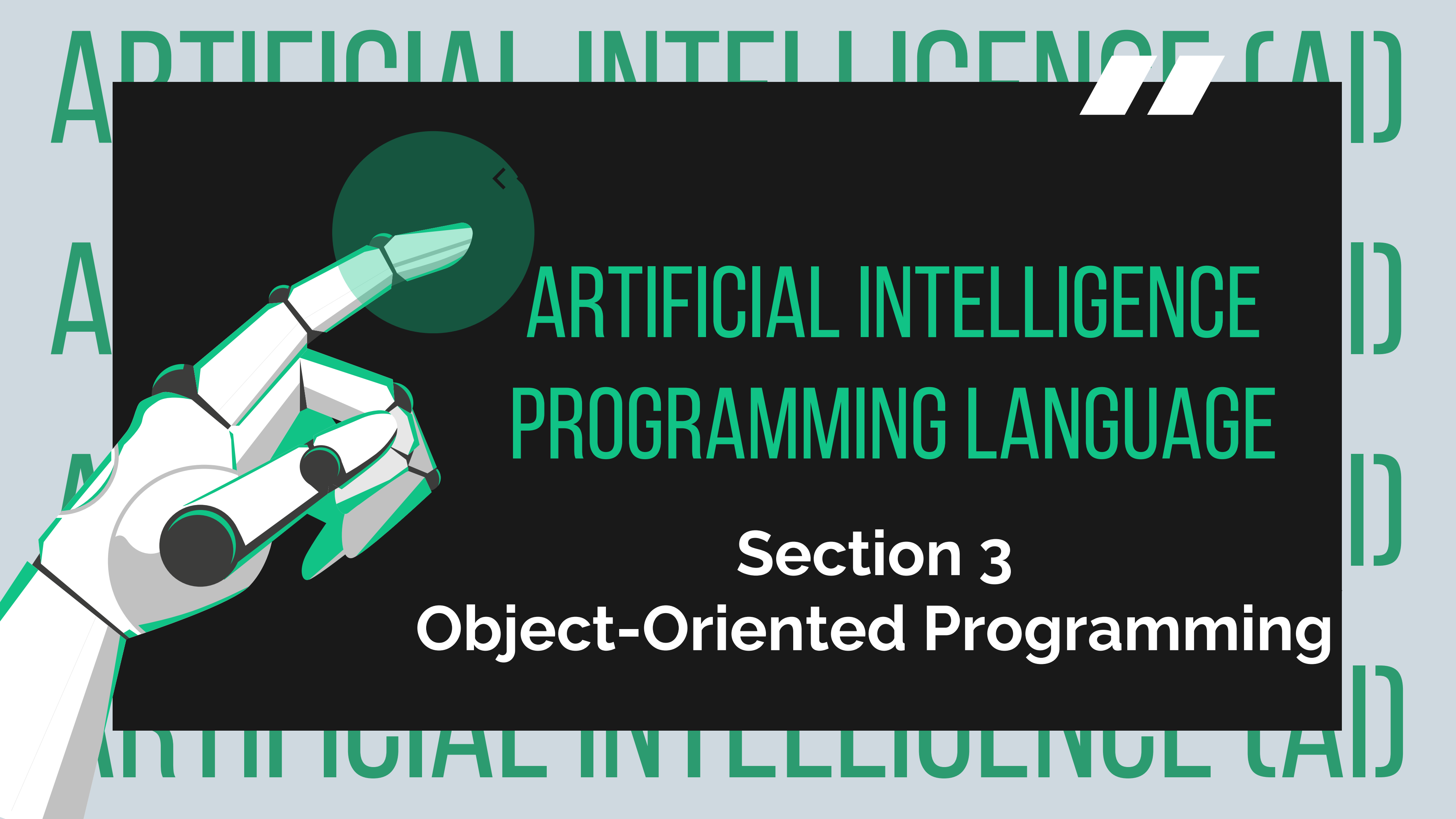




ARTIFICIAL INTELLIGENCE PROGRAMMING LANGUAGE

Section 3 Object-Oriented Programming

WHAT IS OOP?

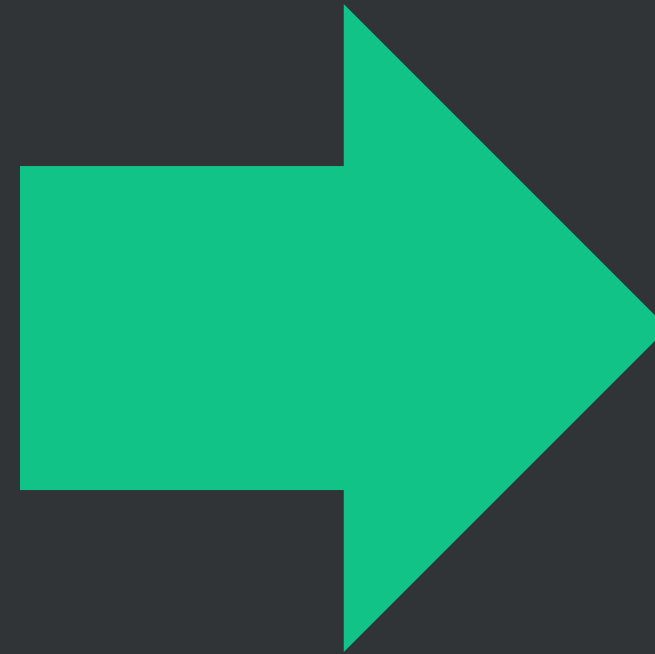
a programming paradigm based on the concept of "objects," which contain both data (attributes) and code (methods). It organizes software design around data rather than functions, making it ideal for managing complex, large-scale software.

Real-Life Example: Car



A CAR HAS:

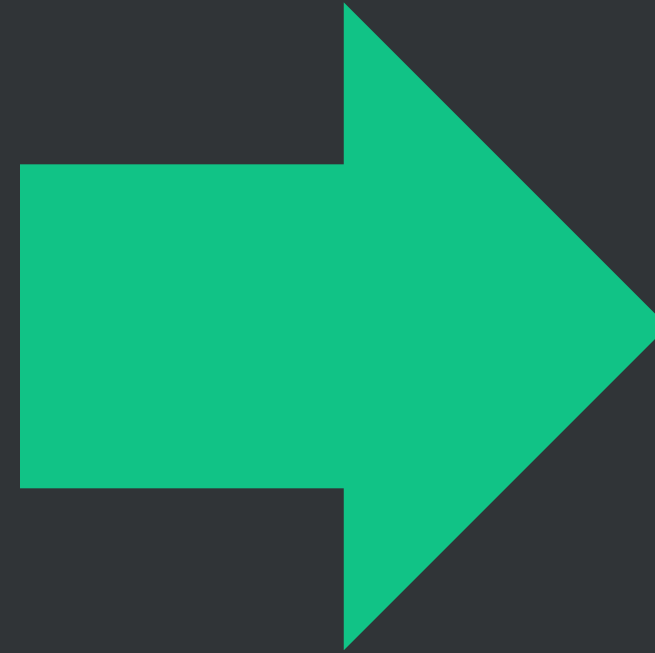
COLOR MODEL SPEED



Attributes

A CAR CAN:

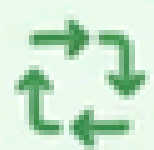
DRIVE STOP ACCELERATE



Methods

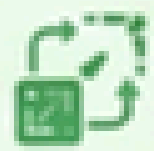
Why Do We Use OOP?

WHY WE USE OOP (Benefits)



Modularity & Reusability

(Building blocks, less code duplication)



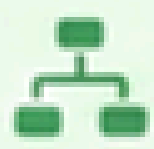
Maintainability & Scalability

(Easier to update and grow large projects)



Data Abstraction & Encapsulation

(Hiding complexity, securing data)



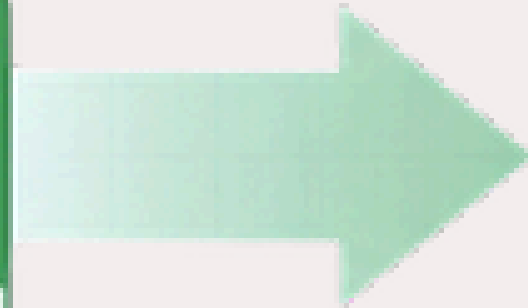
Inheritance & Polymorphism

(Flexible, extensible code structures)



Real-World Modeling

(Intuitive design based on objects)



?



OBJECT-ORIENTED PROGRAMMING (OOP): WHY & WHY NOT



WHAT HAPPENS WITHOUT OOP (Consequences)



Spaghetti Code

(Tangled, hard-to-follow logic)



Difficulty in Maintenance

(Small changes break unexpected parts)



Lack of Reusability

(Rewriting similar code repeatedly)



Poor Scalability

(Projects become unmanageable as they grow)

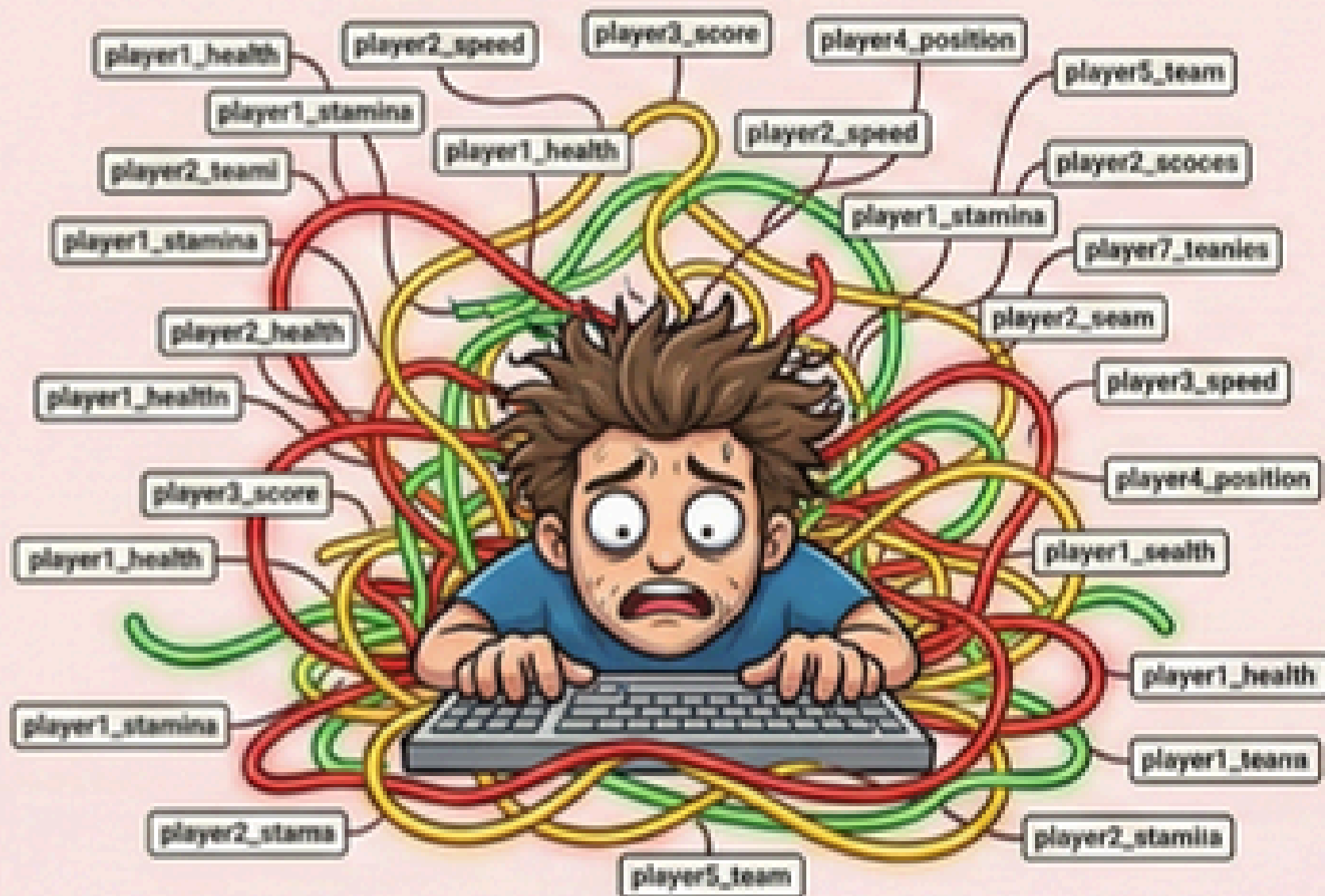


Higher Error Rate

(Increased complexity leads to more bugs)

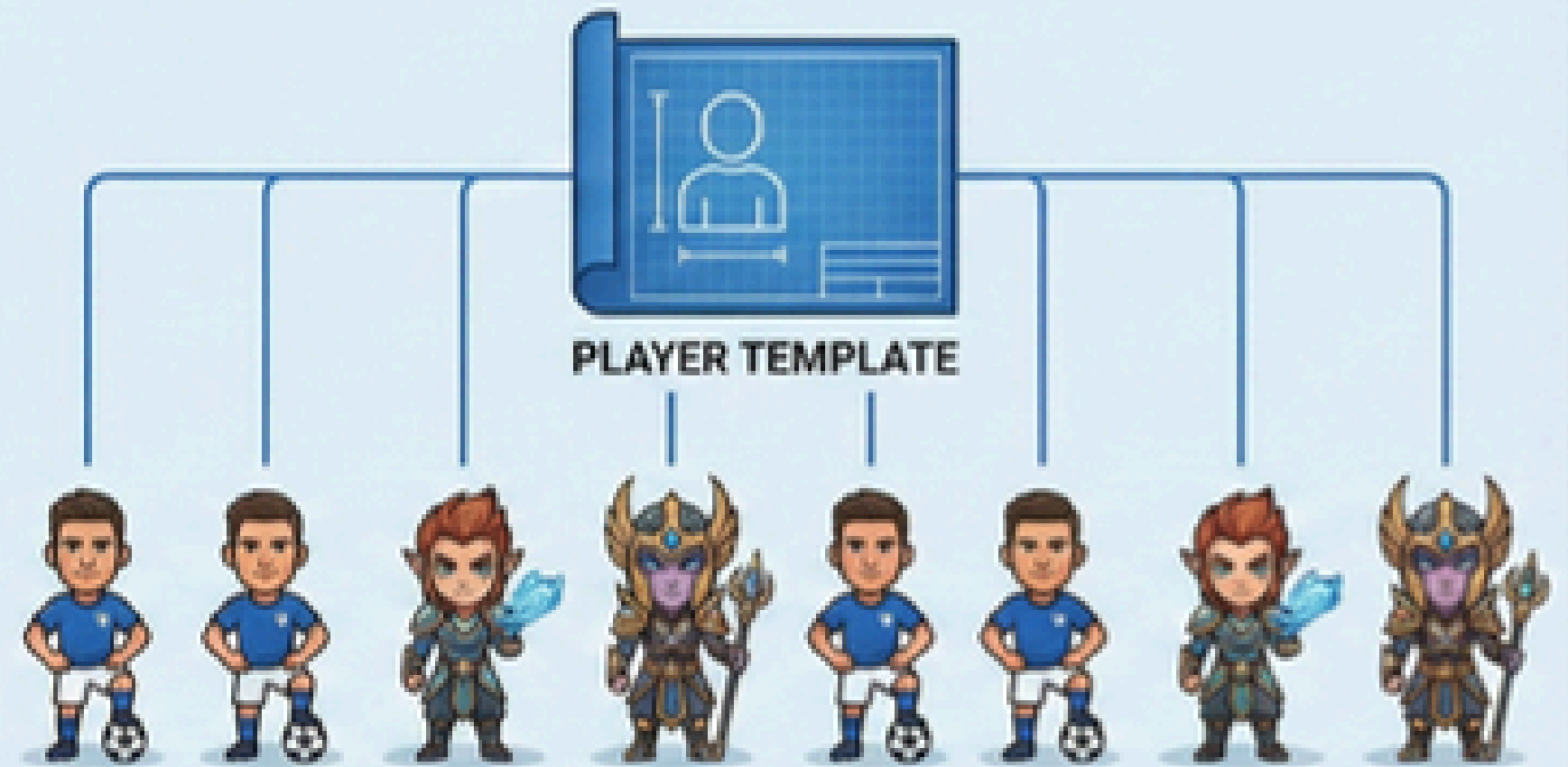
Why Do We Use OOP?

PROCEDURAL APPROACH



Thousands of variables for **EVERY** single player.
MESSY, HARD TO MANAGE, REPETITIVE.

OOP APPROACH



Create **ONE** template (**Player**) and copy it as many times as you want.
ORGANIZED, EFFICIENT, SCALABLE.

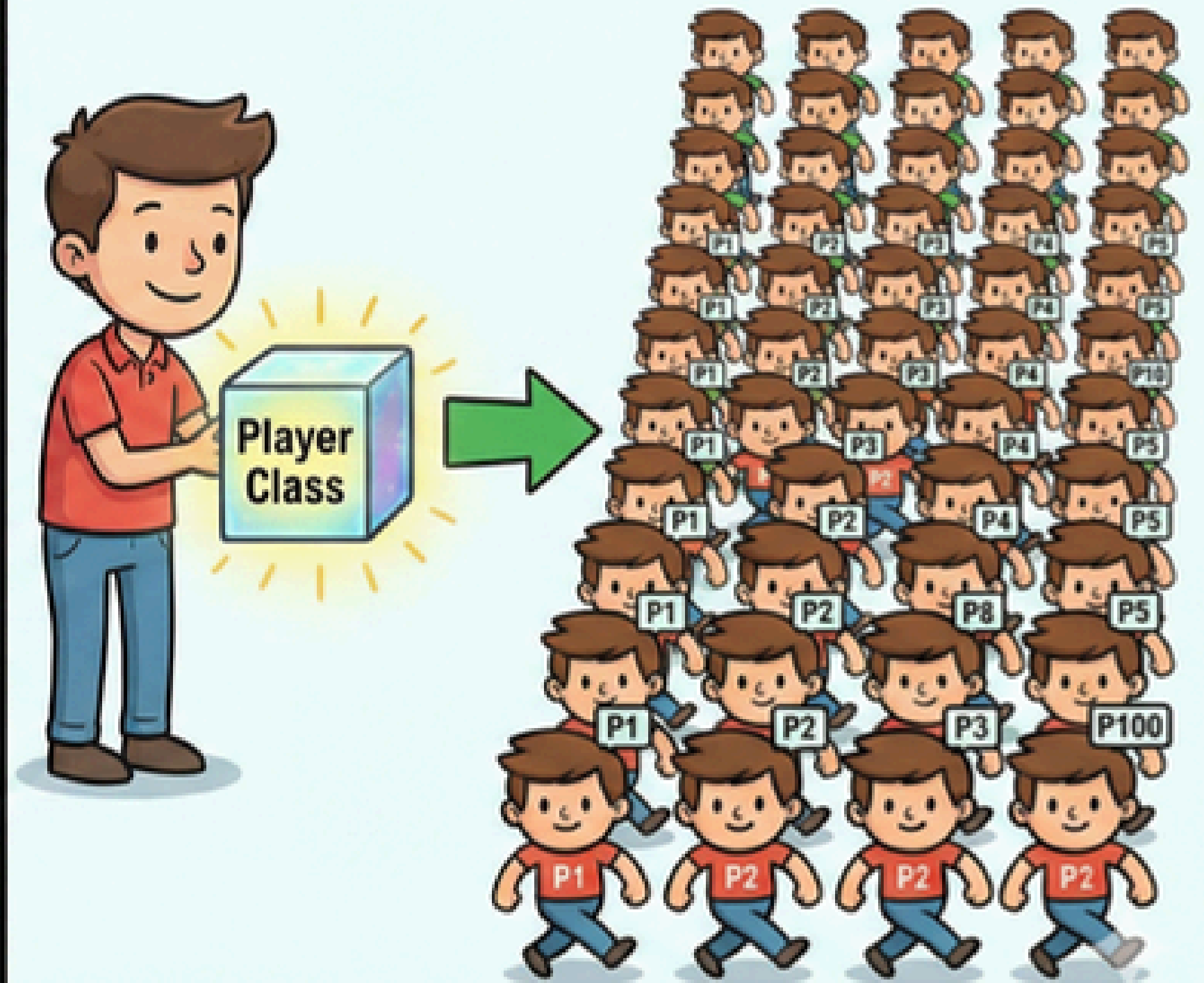
KEY BENEFIT: DRY PRINCIPLE
DON'T REPEAT YOURSELF. 

IF YOU'RE BUILDING A GAME WITH 100 PLAYERS... WILL YOU CREATE 100 SEPARATE VARIABLES?

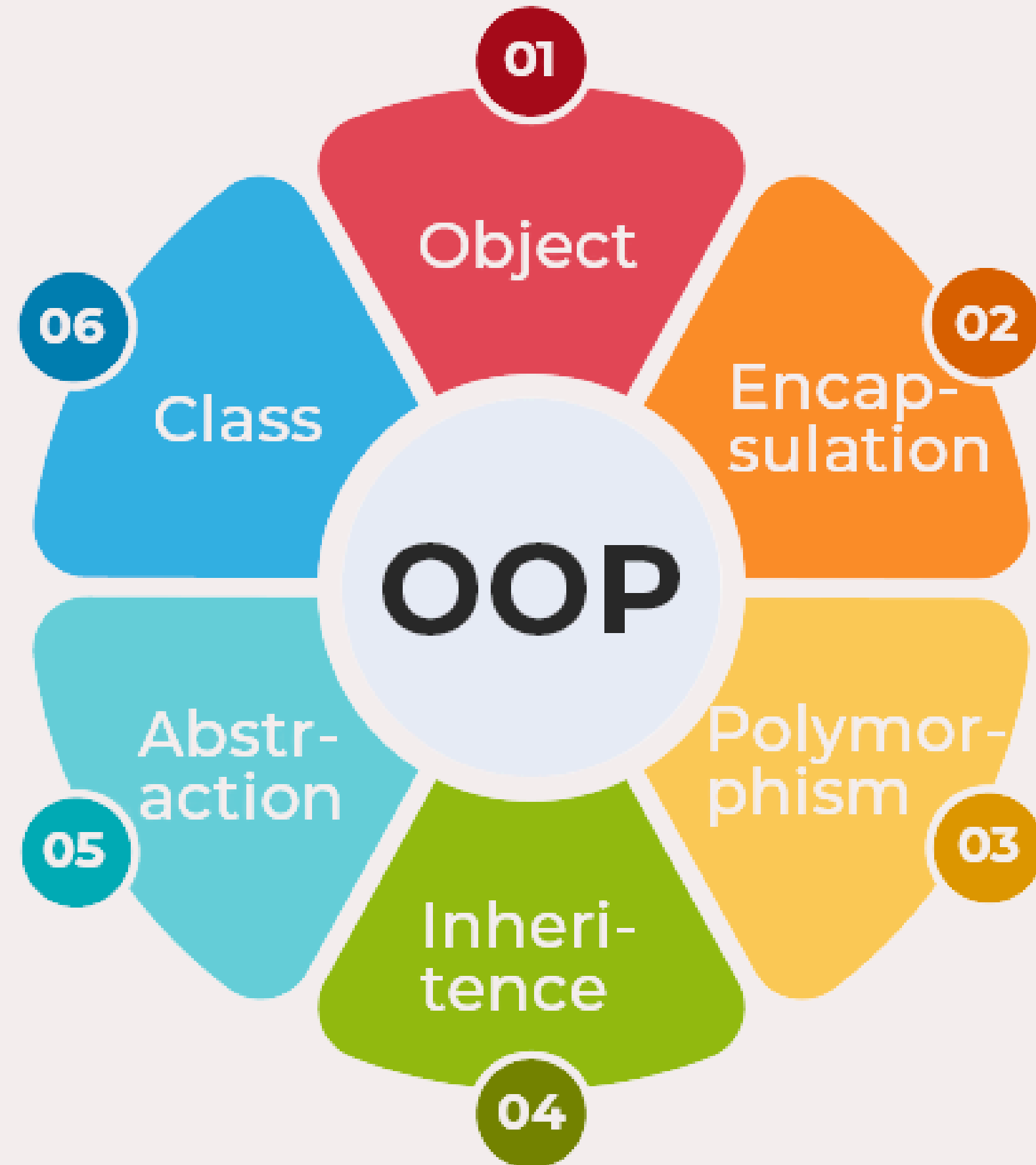
100 VARIABLES?
CHAOS!



1 OBJECT (CLASS) = 100 PLAYERS!
EFFICIENT!



Core OOP Concepts



Core OOP Concepts

1. Attributes (Data/Properties)



COLOR: Blue
HEIGHT: 180cm
NAME: 'Hero'
SPEED: 95

What the object **IS** or **HAS**.

- Color (palette)
- Height (ruler)
- Name (tag)
- Speed (speedometer)

2. Methods (Behavior/Action)



What the object **DOES**.

- Walk() (footsteps)
- Shoot() (target)
- Calculate_Tax() (calculator)
- Login() (lock)

Class vs object

CLASS - The Cookie Cutter



Instantiation

OBJECT - The Cookies



Template
Defines Shape & Ingredients
Only One Exists

Instances
Created from Template
Many Unique Objects Exist

__init__ function (constructor)

How __init__ Works in Memory 🧠

The Trigger
(Call Command)

```
s1 = Student("Ali", 20)
```

<<<<

The Mapping
(Linkage Process)

New Empty Object 

Refers to the new
object being created

```
def __init__(self, name, age):  
    self.name = name  
    self.age = age
```

The Result
(RAM Layout)

Object s1 in Memory

```
name: "Ali"  
age: 20
```

A Simple Bank Account

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited ${amount}. New Balance: ${self.balance}")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient Funds!")
        else:
            self.balance -= amount
            print(f"Withdrew ${amount}. Remaining: ${self.balance}")

# Usage
my_account = BankAccount("Ahmed", 1000)
my_account.deposit(500) # Balance: 1500
my_account.withdraw(2000) # Insufficient Funds!
```

Online Shopping Class

```
class Product:
    def __init__(self, name, price, quantity):
        self.name = name          # Store the product name
        self.price = price        # Store the product price
        self.quantity = quantity  # Store the product quantity

    def total_price(self):
        return self.price * self.quantity  # Calculate total price (price × quantity)

# Create an object (product)
item1 = Product("Laptop", 20000, 2)
# name = "Laptop"
# price = 20000
# quantity = 2
|
# Call the method to calculate total price
print("Total:", item1.total_price())
# total_price() = 20000 × 2 = 40000
# Final Output:
# Total: 40000
```


What is *self*?

The "Identity Card" of the Object

Definition: *self* is a reference to the current instance of the class. It is how the object refers to itself.

The Problem: When you create 100 objects from one class (e.g., 100 Students), how does Python know which student's name you want to print?

The Solution: *self* tells Python: "I am talking about THIS specific object's data, not the others."

Rule: It must be the first parameter of any method within a class. <<<<

What is self?

The "Identity Card" of the Object
self is like saying "MY"

Scenario: Imagine a room full of people.

Without self: If you shout "What is the name?", everyone is confused. Which name?

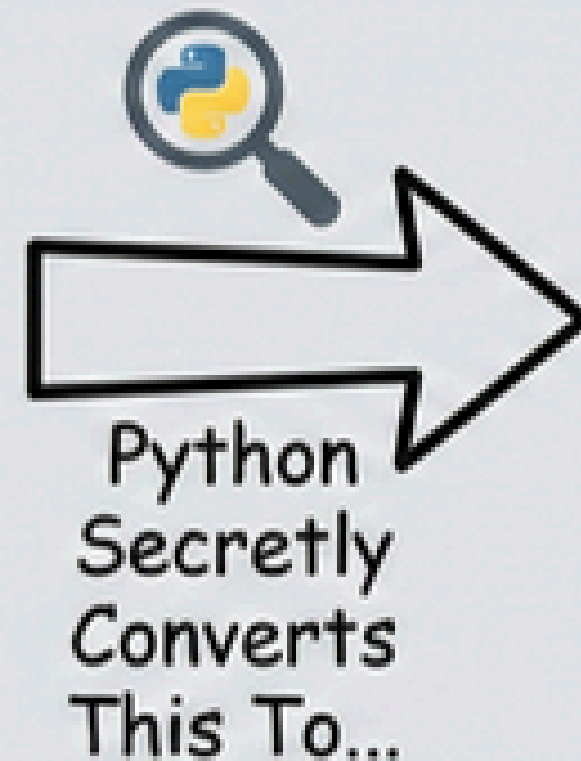
With self: If you ask a specific person "What is YOUR (self) name?", only that person answers with their own name.



How self works?

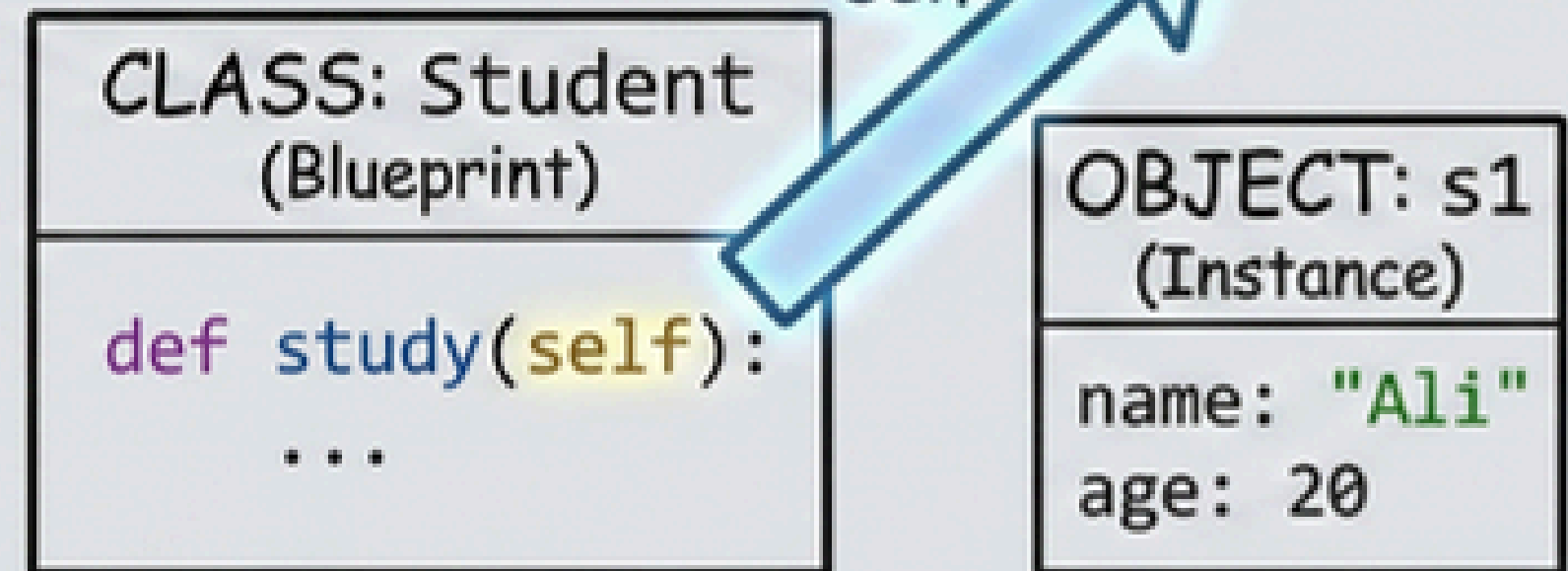
THE CODE YOU WRITE

```
s1.study()
```



THE ACTUAL CALL (Behind the Scenes)

```
Student.study(s1)
```



The "s1" goes into "self". It's the link that connects the code in me in the Class to the data in the Object.

What is self?

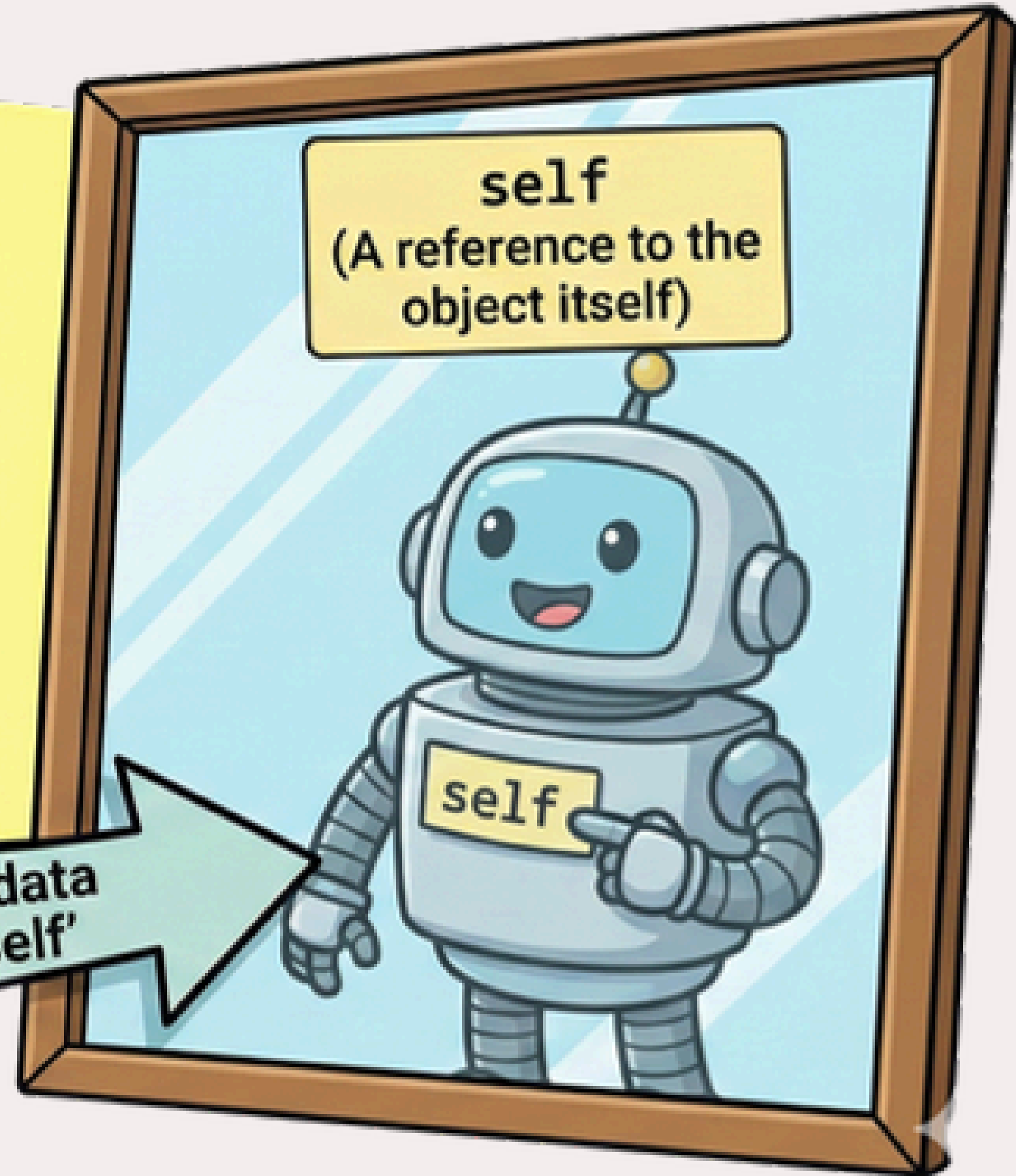
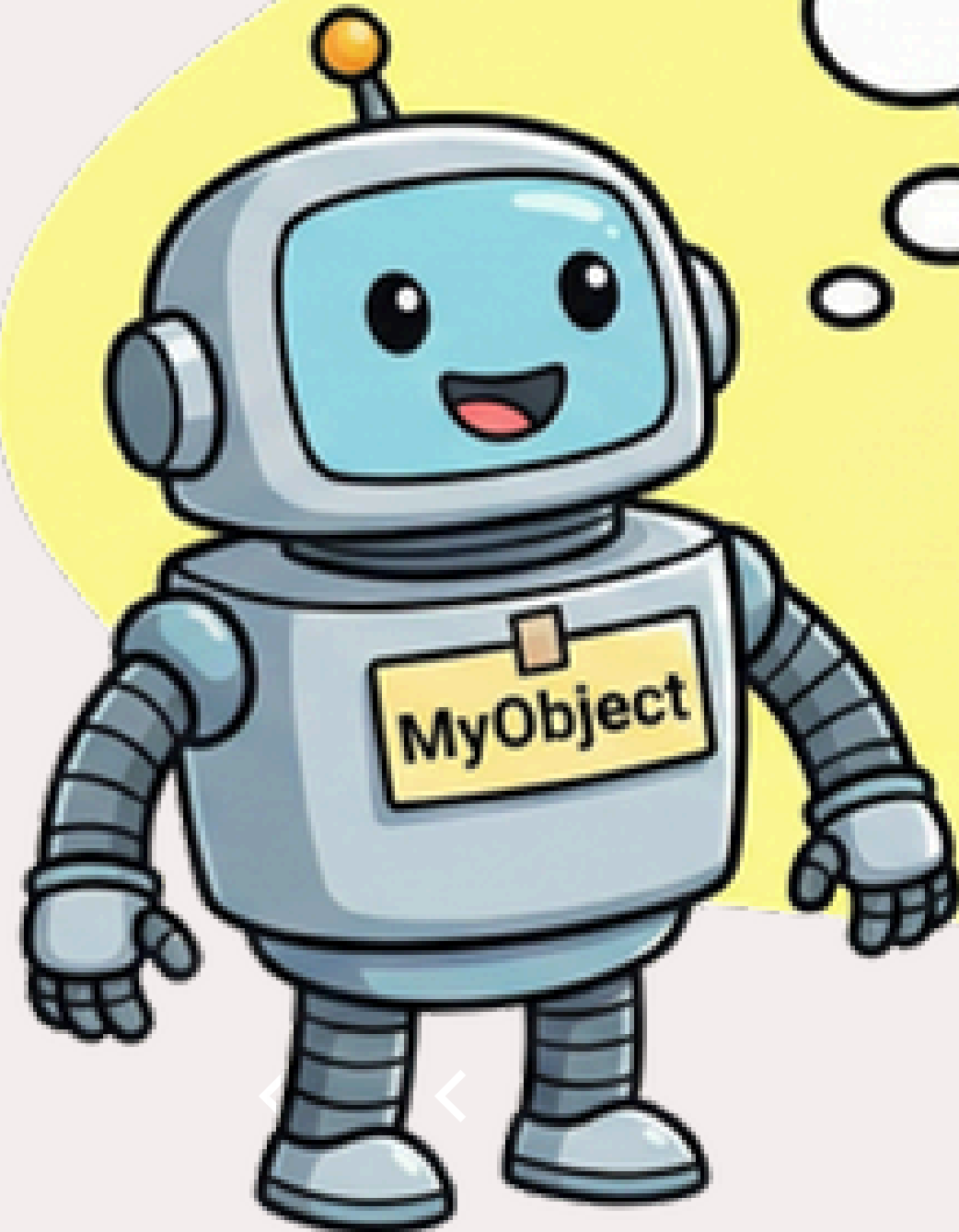
```
self.name = 'MyObject'  
self.do_something()
```

MyObject

Accessing its own data
and methods via 'self'

self
(A reference to the
object itself)

self



Inheritance

Base Class: Employee

"Every single person in the company has these basic traits.
We write this ONCE."

```
class Employee:
    def __init__(self, name, emp_id, salary):
        self.name = name
        self.emp_id = emp_id
        self.salary = salary

    def check_in(self):
        print(f"✅ {self.name} (ID: {self.emp_id}) has arrived at work.")

    def show_details(self):
        print(f"📄 Employee: {self.name} | Salary: ${self.salary}")
```


Inheritance

The Manager Class

"A Manager is an Employee, but they manage a team."

- Extra Attribute: `team_size` (Number of people).
- Extra Method: `hold_meeting()` (Only managers do this).

```
class Manager(Employee):
    def __init__(self, name, emp_id, salary, team_size):
        # Pass common data to Parent
        super().__init__(name, emp_id, salary)
        # Handle specific data
        self.team_size = team_size

    def hold_meeting(self):
        print(f"🔊 {self.name} is holding a meeting with {self.team_size} people.")
```

Inheritance

The Developer Class

“A Developer is an Employee, but they write code.”

- Extra Attribute: `prog_lang` (Python, Java, C++).
- Extra Method: `fix_bug()` (Only devs do this).

```
class Developer(Employee):
    def __init__(self, name, emp_id, salary, prog_lang):
        super().__init__(name, emp_id, salary)
        self.prog_lang = prog_lang

    def fix_bug(self):
        print(f"🐛 {self.name} is fixing a bug in {self.prog_lang}...")
```

Inheritance

The Office Boy Class

"An Office Boy is an Employee, but they manage supplies."

- Extra Attribute: shift (Morning/Night).
- Extra Method: serve_coffee() (Specific task).

```
class OfficeBoy(Employee):
    def __init__(self, name, emp_id, salary, shift):
        super().__init__(name, emp_id, salary)
        self.shift = shift

    def serve_coffee(self):
        print(f"☕ {self.name} is serving coffee during the {self.shift} shift.")
```


Inheritance

The Main Code

```
# 1. Create the Staff
m1 = Manager("Mr. Ahmed", 101, 9000, 15)
d1 = Developer("Sarah", 102, 5000, "Python")
o1 = OfficeBoy("Ali", 103, 2000, "Morning")

print("--- Morning Routine ---")

# 2. Everyone Checks In (Inherited Method)
m1.check_in()
d1.check_in()
o1.check_in()

print("\n--- Working Time ---")

# 3. Everyone does their specific job (Unique Methods)
m1.hold_meeting()    # Manager specific
d1.fix_bug()         # Developer specific
o1.serve_coffee()    # Office Boy specific
```

Inheritance

What is super()?

Definition: `super()` is a built-in function that allows a Child Class to call methods from its Parent Class.

The Goal: It gives you access to the parent's "tools" without rewriting them.

Most Common Use: Inside the `__init__` method.

We use it to let the Parent handle the basic setup (name, id), while the Child focuses on the new stuff.

Inheritance

What is `super()`?

Without `super()`: If you define `__init__` in the Child, you completely overwrite (kill) the Parent's `__init__`. The Child forgets everything the Parent knew.

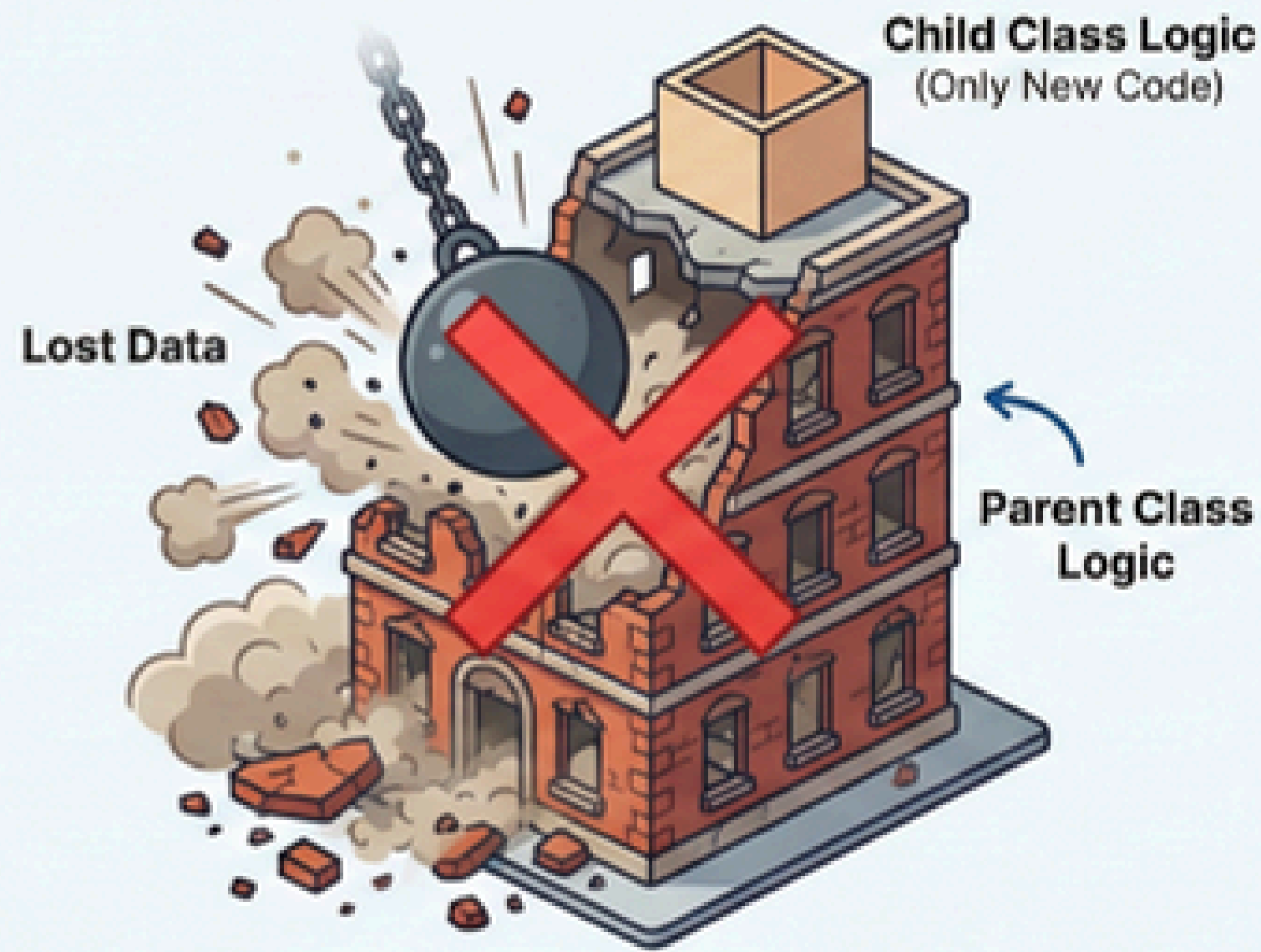
With `super()`: You are saying:

1. "First, run the Parent's code." (Keep the old logic).
2. "Now, run my new code." (Add the new logic).

Inheritance

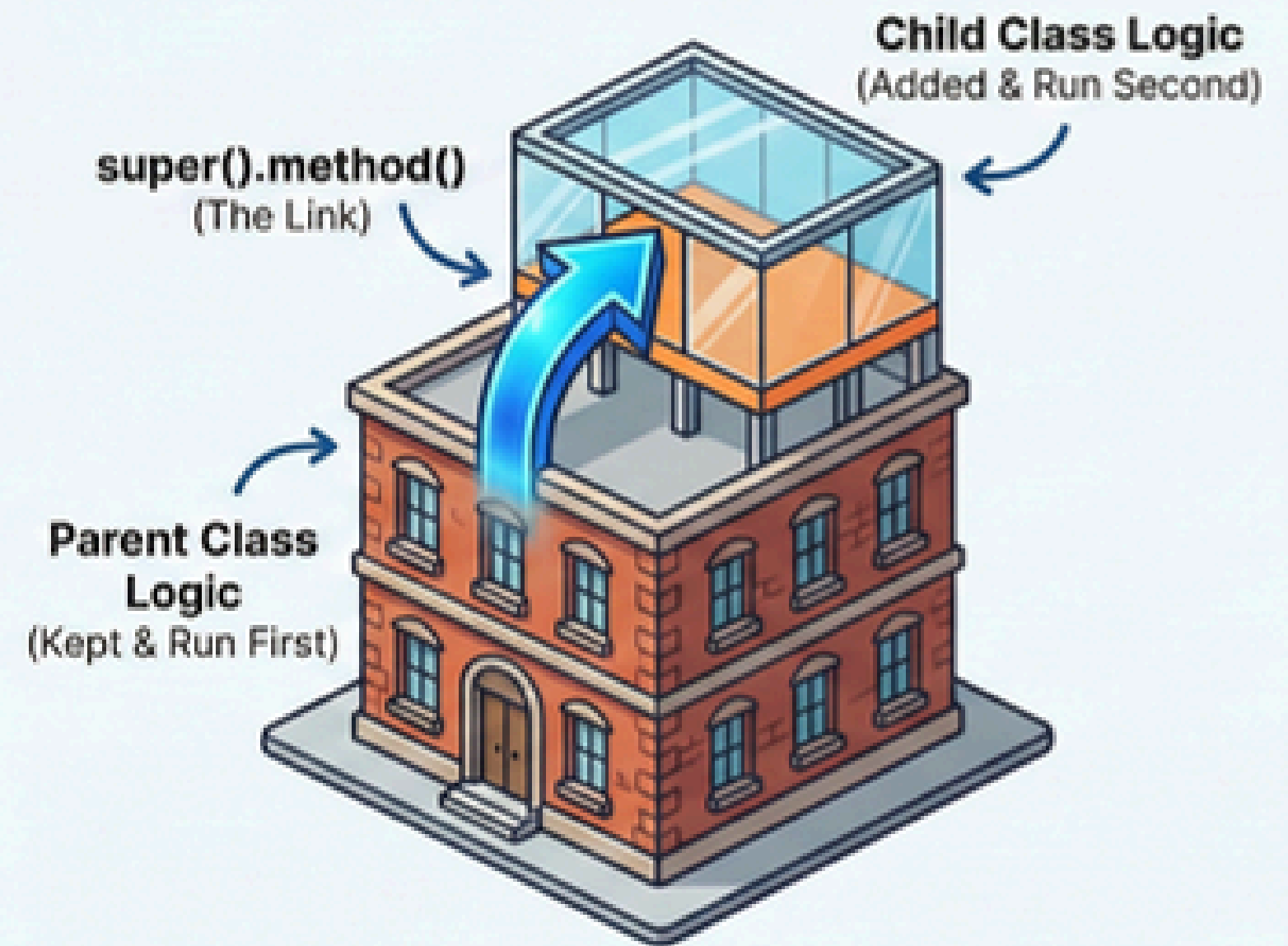
The Role of `super()`: Extension, Not Replacement 🚀

WITHOUT `super()` (Overwriting)



**Destroys Parent's Logic.
Loses Old Data.**

WITH `super()` (Extending)



**Keeps Parent's Logic. Adds New Data.
Extends Functionality.**

Let's code



CHALLENGE : THE SECURE ATM

CLASS & ATTRIBUTES

- Create class `ATM`
- `balance` (float, starts at 1000.0)
- `pin` (int, e.g., 1234)

METHODS

- `withdraw(amount, input\_pin)`:
 - If `input\_pin` != `self.pin`: Print "❌ Wrong PIN!"
 - If `amount` > `self.balance`: Print "❌ Insufficient Funds!"
 - If both OK: Subtract `amount`, Print "✅ Withdraw successful! New Balance: \${X}"

TEST CASE

1. Try to withdraw \$500 with a wrong PIN.
2. Try to withdraw \$500 with the correct PIN.



Secure ATM

```
class ATM:
    def __init__(self, initial_balance=1000, pin=1234):
        self.balance = initial_balance
        self.pin = pin

    def withdraw(self, amount, input_pin):
        # Requirement 1: Check PIN first
        if input_pin != self.pin:
            print("✗ Wrong PIN!")
            return # Exit the method early

        # Requirement 2: Check funds
        if amount > self.balance:
            print("✗ Insufficient Funds!")
            return # Exit the method early

        # Requirement 3: Process transaction
        self.balance -= amount
        print(f"✓ Withdraw successful! New Balance: ${self.balance}")
```

```
# --- Test Case ---

my_atm = ATM()

# 1. Try to withdraw $500 with a wrong PIN
print("Attempt 1 (Wrong PIN):")
my_atm.withdraw(500, 9999)

print("\nAttempt 2 (Correct PIN):")
# 2. Try with the correct PIN
my_atm.withdraw(500, 1234)
```



CHALLENGE: The 'Netflix' Content Manager 🎬

THE MISSION

PARENT CLASS: Content

- Attributes: title (str), genre (str), views (int)
- Method: watch() ➡ Increases views by 1

CHILD CLASSES (INHERITANCE)

CLASS: Movie 🎬

- Unique Attribute: duration (int, mins)
- Method Override: get_info() ➡ Prints "Movie: [Title] | Duration: [X] mins"

CLASS: Series 📺

- Unique Attribute: episodes (int)
- Method Override: get_info() ➡ Prints "Series: [Title] | Episodes: [X]"

YOUR TASK 📝

1. Create a list called "my_watchlist".
2. Add one Movie object and one Series object to it.
3. Loop through the list and call "get_info()" for each item. ✨

"Netflix" Content Manager

```
class Content:
    def __init__(self, title, genre):
        self.title = title
        self.genre = genre
        self.views = 0

    def watch(self):
        self.views += 1

class Movie(Content):
    def __init__(self, title, genre, duration):
        super().__init__(title, genre)
        self.duration = duration

    def get_info(self):
        print(f"Movie: {self.title} | Duration: {self.duration} mins")

class Series(Content):
    def __init__(self, title, genre, episodes):
        super().__init__(title, genre)
        self.episodes = episodes

    def get_info(self):
        print(f"Series: {self.title} | Episodes: {self.episodes}")
```

```
# --- Task ---
my_watchlist = [
    Movie("Inception", "Sci-Fi", 148),
    Series("Stranger Things", "Horror", 34)
]

for item in my_watchlist:
    item.get_info()
```

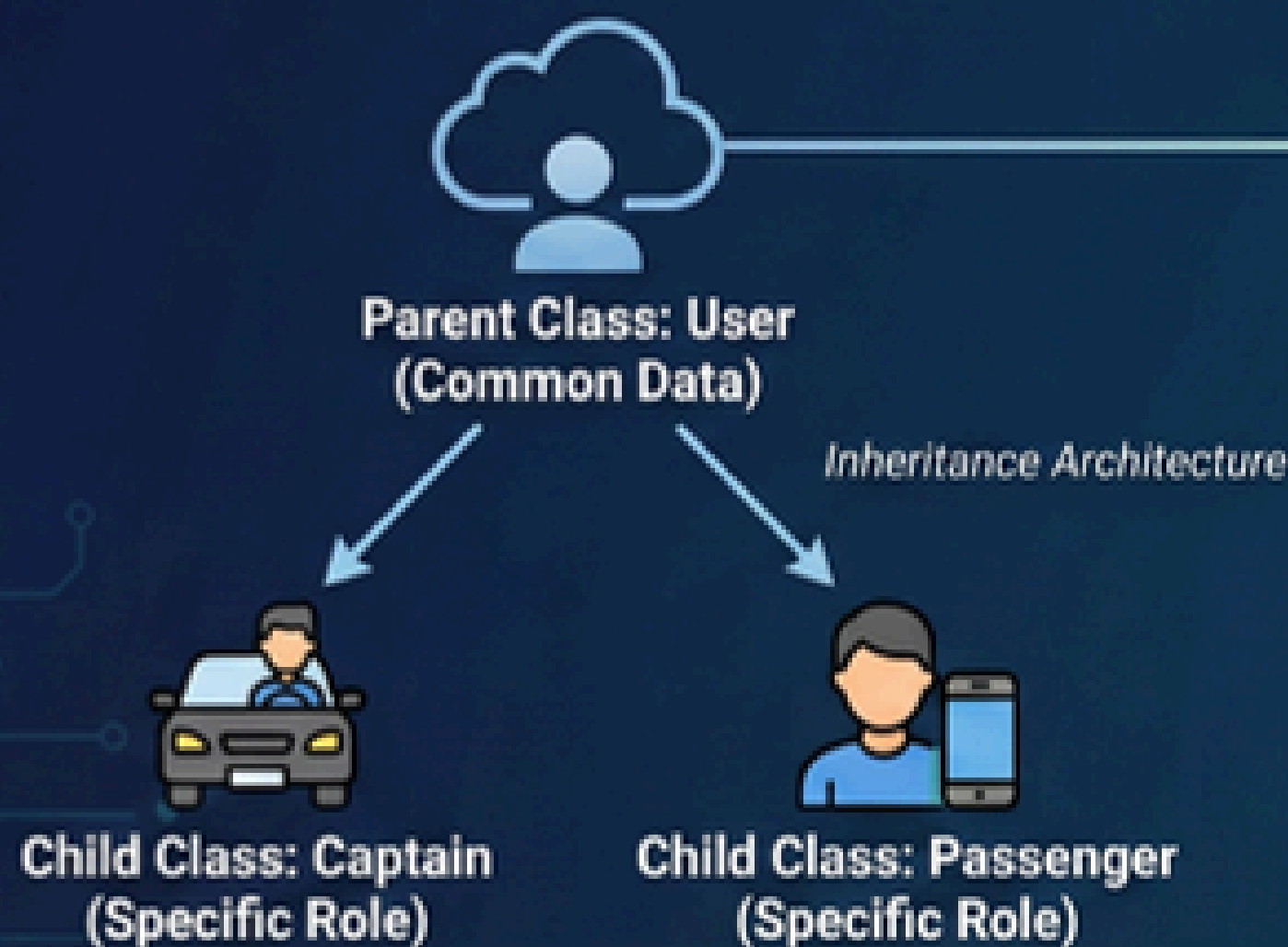
Assignment

🚀 FINAL CHALLENGE: CAREEM/UBER SYSTEM 🚗 – THE MISSION & REQUIREMENTS

THE SCENARIO: RIDE-SHARING BACKEND ENGINEER 🚀

Your Role & Task:

- 👤 Lead Backend Engineer at a startup.
- 🔧 Design core system for Captains (Drivers) & Passengers.
- 🎯 Goal: Create a scalable, object-oriented architecture.



SYSTEM REQUIREMENTS & ARCHITECTURE 📋

1. PARENT CLASS: USER (Common Data)

Attributes:

- 👤 name (String)
- 📞 phone (String)
- ✉ email (String)

Method:

- 🔑 login(): Prints 'Welcome back, [Name]!'

2. CHILD CLASS: CAPTAIN (Inherits from User)

Unique Attributes:

- 🚗 car_model (e.g., 'Nissan Sunny')
- 💰 balance (Starts at 0.0)
- ★ rating (Starts at 5.0)

Unique Method:

- ✅ complete_trip(amount): Adds amount to balance, prints "Trip finished! New Balance: \$[X]".

3. CHILD CLASS: PASSENGER (Inherits from User)

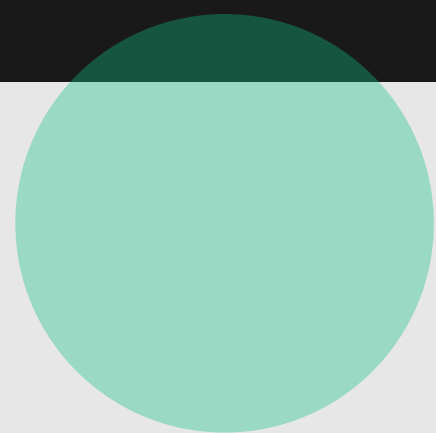
Unique Attributes:

- 📍 location (String)
- 💳 payment_method (e.g., 'Cash' or 'Visa')

Unique Method:

- 📱 request_ride(destination): Prints "Looking for a captain to go to [Destination]..."

/CAID/CAID/



THANKS

ARTI
IAL
INTER
GENO
CAID